Loan Lens:

Lending Club Credit Risk Analytics Platform

Finance | Lending Club Loan Data (loan.csv)

Ajmiirah Eadally

Rukhsaar Baichoo

Chrishni Mungada

KAVINA NARRAINEN

Polytechnics Mauritius Ltd

27/02/2026

Table of Contents

1. INTRODUCTION 4

1.1 Project Overview 4

1.2 Problem Statement 4

2. STAKEHOLDERS & KPI 5

2.1 Stakeholders 5

2.2 Stakeholder Questions 5

2.3 Key Performance Indicators (KPIs) 6

3. DATASET SELECTION, PROFILING & PREPARATION 8

3.1 Dataset Size and Structure 8

3.2 Dataset Profiling & Cleaning section 8

3.2.1 Missing fields 8

3.2.2 Inconsistent values 9

3.2.3 Duplicate check 9

3.2.4 Structural and Header Cleaning 9

3.2.5 Transformations Performed in ETL 10

3.2.6 Sampling Justification 10

Big Data Architecture Flow 11

4.1 Data Source 11

3.3 Ingestion 11

3.4 Storage in HDFS 12

4.3.1 HDFS Folder Structure 12

4.3.2 Evidence of Storage 12

4.4 Processing / ETL (Hadoop Tool: Apache Pig) 13

4.5 Architecture Diagram 14

Serving Layer & MongoDB Implementation 16

5.1 Introduction 16

5.2 Technical Justification for MongoDB Implementation 16

5.3 Data Preparation and Query Development 17

Why not 100,000 rows in CSV? 20

Analytics and Visualization 21

6.1 KPI Definitions 21

6.2 Descriptive Analytics 22

6.2.1 Default Rate Distribution by Loan Grade 22

6.2.3 Risk Indicator Summary (KPIs 4-6) 23

6.3 Segmentation Analysis 23

6.3.1 Grade-Based Segmentation 23

6.3.2 Purpose-Based Segmentation 23

6.3.3 Term-Based Segmentation 23

6.4 Trend Analysis 24

6.5 Deeper Insight - Interest Rate vs Default Rate Correlation 24

Visualization & Dashboard 25

7.1 Dashboard Overview Screenshots 25

7.2 Technology Choice Justification 25

7.3 Interactive Filters 26

7.4 Chart-by-Chart Description 26

7.5 Insights & Recommendations Panel 29

7.5 Alignment to Stakeholder Questions 30

7.6 Dashboard Submission & Reproduction 31

Issues Encountered and Solutions 31

Project Management & Team Contribution 32

Jira 32

9.2 Github 36

References 37

Appendix A - Team Reflection 38

1. INTRODUCTION

1.1 Project Overview

This project presents the design and implementation of an end-to-end Big Data analytics platform using the Lending Club loan dataset. The objective is to build a complete data pipeline that ingests raw data into Hadoop Distributed File System (HDFS), processes it using Apache Pig, serves structured outputs via MongoDB, and delivers actionable insights through a KPI-based interactive dashboard.

The architecture follows a layered approach:

1. Raw data ingestion into HDFS
2. Data cleaning and transformation using Apache Pig
3. Storage of curated datasets in HDFS
4. Loading processed outputs into MongoDB
5. KPI computation and aggregation
6. Dashboard visualization and stakeholder insight generation

The platform demonstrates a realistic enterprise-grade Big Data architecture.

1.2 Problem Statement

Consumer lenders must balance credit risk with stable growth in loans and returns. The key issue here is that some consumers will just stop paying their loans, and this results in defaults and charge-offs that negatively affect portfolio performance. One of the methods used to determine credit risk is the default rate, which is the percentage of loans that are not repaid and are considered in default.

1. It determines which segments of the loans have higher risk based on the patterns of outcome in the loan_status variable.
2. It verifies if the pricing signals, such as the interest rate, are consistent with the risk observed in the segments.
3. It tracks changes in risk and performance over time.

2. STAKEHOLDERS & KPI

2.1 Stakeholders

The primary stakeholders for this project are:

1. **Credit Risk Manager:** This stakeholder is responsible for monitoring the portfolio's default risk and determining which segments of borrowers are highly risky.
2. **Pricing/Product Manager:** This stakeholder is responsible for ensuring that the interest rates for loans are in sync with the observed risk.
3. **Portfolio Performance Analyst:** This stakeholder is responsible for tracking trends in loan performance and segment-level performance over time.
4. **Underwriting Policy:** This stakeholder is responsible for determining the criteria for approving borrowers based on risk indicators.

2.2 Stakeholder Questions

The project is structured to answer the following key decision-oriented questions:

1. **What segments of the loan portfolio have the highest credit risk** based on the percentage of loans ultimately having negative outcomes (e.g., Charged Off or Default), grouped by grade, term, purpose, and DTI bands?
2. **Are the prices of the loans aligned with the risk?** In other words, do segments with higher average interest rates tend to have higher default or charge-off percentages across all grades and terms?
3. **How does risk change over time?** Are there discernible patterns in loan outcomes by issue month or year?
4. **What characteristics do borrowers or loans tend to have the highest association with negative outcomes?**
5. **What segments of the portfolio tend to have the best overall performance** based on repayment and simple return/loss calculations?

These questions assure that the analytics layer is decision-oriented and thus has direct value to the stakeholders as opposed to merely being of descriptive interest.

2.3 Key Performance Indicators (KPIs)

To address these questions from the stakeholders, the following KPIs are defined and computed based on the processed dataset.

KPI 1: Default/Charge-Off Rate

Definition: Proportion of loans with negative credit performance.

Formula: Default Rate = Number of loans where loan_status ∈ {Charged Off, Default} / Number of loans with final outcomes

Where final outcomes include: Fully Paid**,** Charged Off**,** Default

Loans with status "Current" or other active states are not included in this formula to avoid bias.

Why it matters: This is the primary credit risk indicator. This measures realized portfolio risk and allows for a comparison of the risk across grades, terms, purposes, states, and borrower characteristics.

KPI 2: Fully Paid Rate

Definition: Proportion of loans with successful repayment in full.

Formula: Fully Paid Rate = (Number of loans with loan_status = Fully Paid) / Number of loans with final outcomes

Why it matters: This KPI is a complementary metric to the Default Rate and is a measure of the success ratio for the lending portfolio.

KPI 3: Average Interest Rate

Definition: Average interest rate applied to a specific segment.

Formula: Average Interest Rate = AVG(int_rate)

Why it matters: This KPI captures loan pricing level. By comparing it with Default Rate, risk-based pricing alignment can be assessed.

KPI 4: High Debt-to-Income (DTI) Risk Share

Definition: Percentage of loans where debt burden exceeds a certain ratio.

Example Formula (DTI > 30%):

High DTI Share = Number of loans where dti > 30 / Total number of loans

Why it matters: High DTI ratio is a sign of financial distress and correlates with a higher probability of default. This KPI helps with risk segmentation of borrowers.

KPI 5: Revolving Utilisation Risk Indicator

Definition: Measurement of credit usage intensity from a borrower's perspective.

Two options to compute this KPI:

Option A - Average Utilisation: AVG(revol_util)

Option B - High Rev Utilisation Share (> 80%):

Number of loans where revol_util > 80 / Total loans

Why it matters: Revolving credit utilisation at high levels indicates a strong dependence on credit. This has been statistically correlated to higher default risks.

KPI 6: Delinquency Risk Share

Definition: Proportion of loans from borrowers who have at least one delinquency in the last two years.

Formula: Delinquency Share = Number of loans where delinq_2yrs > 0 / Total number of loans

Why it matters: Delinquency in the past two years is a strong predictor of default in the future.

3. DATASET SELECTION, PROFILING & PREPARATION

The data source is the public Lending Club loan dataset (loan.csv) downloaded from Kaggle. This is appropriate for the project because it includes the loan_status, which offers an outcome measure that facilitates the evaluation of performance and credit risk by comparing across segments such as grade, term, purpose, and time, as opposed to merely describing the borrower.

3.1 Dataset Size and Structure

The raw dataset was ingested into HDFS at this path: /loan_project/raw/loan.csv

Raw file size (HDFS): ~1.1 GB

The dataset record count was verified directly from HDFS to avoid spreadsheet tool limitations. The line count of the raw CSV in HDFS is 2,260,669 lines, including one header row. Therefore, the dataset contains: 2,260,668 number of rows and 146 number of columns. This satisfies the assignment requirement for a large dataset (≥ 500,000 records or ≥ 1 GB).

Evidence commands used:

1. Confirms the raw dataset exists in HDFS and shows its size.

```
rukhsaar@Rukhsaar:~$ hdfs dfs -put -f ~/loan.csv /loan_project/raw/
rukhsaar@Rukhsaar:~$ hdfs dfs -ls -h /loan_project/raw
Found 1 items
-rw-r--r--   1 rukhsaar supergroup      1.1 G 2026-02-17 12:55 /loan_project/raw/loan.csv
rukhsaar@Rukhsaar:~$ hdfs dfs -du -h /loan_project/raw
1.1 G  1.1 G  /loan_project/raw/loan.csv
```

2. Removes the header and returns the actual number of records.

```
rukhsaar@Rukhsaar:~$ hdfs dfs -cat /loan_project/raw/loan.csv | tail -n +2 | wc -l
2260668
```

3.2 Dataset Profiling & Cleaning section

3.2.1 Missing fields

During profiling, several columns were observed to contain missing or blank values. This was particularly visible in: (emp_title, emp_length, revol_util, mths_since_last_delinq, mths_since_last_record, certain secondary credit-related attributes). In the raw dataset, missing values appear either as; empty fields between commas, or explicit values such as "n/a".

Cleaning Rule Applied:

- Missing values were preserved in the curated dataset.
- They were not artificially imputed or replaced.
- Analytical KPIs were calculated only on relevant non-null fields where required (e.g., DTI calculations only applied where dti was present).

3.2.2 Inconsistent values

Some formatting irregularities were found during profiling:

1. The interest rate (int_rate) and revolving utilization (revol_util) were represented as percentage string values (e.g., "13.56%").
2. The loan term values were represented as text string values (e.g., "36 months," "60 months").
3. The loan_status attribute included several outcome statuses, including: (Fully Paid, Charged Off, Default, Current)

Cleaning and Normalization Rules Applied:

1. Percentage attributes were changed to numeric format by stripping the % sign to facilitate numeric grouping.
2. The term values were retained but normalized for grouping.
3. For risk evaluation purposes, only the final outcome statuses were used in the calculation of Default Rates:
    a. **Included:** Fully Paid, Charged Off, Default
    b. **Excluded:** Current and active loan.

3.2.3 Duplicate check

A structural duplicate check was performed at the record level. The dataset does not include repeated full-row duplicates based on the original CSV content. No duplicate removal operation was required during the ETL stage. The curated dataset preserves the same number of records as the raw dataset, except for malformed rows filtered during sanitation

3.2.4 Structural and Header Cleaning

The following structural cleaning steps were implemented:

1. The header row was removed when generating the curated dataset to ensure compatibility with Pig processing.
2. The raw file was preserved intact in the raw zone.
3. A clean header file was optionally stored separately for reference.
4. Rows with mismatched column counts were filtered during sanitation to ensure consistent schema structure (146 columns per record).

3.2.5 Transformations Performed in ETL

The following transformations were applied in the curated/ETL layer:

1. Casting of numeric fields (loan amount, interest rate, DTI, revolving utilization).
2. Creation of grouping-ready fields for segmentation (e.g., grade, term, purpose).
3. Extraction or grouping of issue dates for time-based trend analysis.
4. Filtering of final outcome statuses for default rate computation.

No synthetic features or predictive modelling variables were added at this stage. The transformations were limited to normalization, casting, and aggregation preparation.

3.2.6 Sampling Justification

Though the raw data was stored in HDFS as required, sampling was incorporated during the curation process due to observed discrepancies in data quality after the data was exported. While examining the cleaned data set, it was noticed that some rows contained inconsistent numbers of columns (schema inconsistencies), which cannot be manually corrected on a scale.

A **random sample of 100,000 records** was thus created from the cleaned data set to facilitate manual examination and validation of cleaning rules, while still maintaining representativeness of the data set. Furthermore, a sample data set enhances the responsiveness of the dashboard and facilitates exploratory analysis and KPI development on limited local infrastructure.

4. Big Data Architecture Flow

In this part: Big Data Architecture Flow, describes how the data flows through the key layers of the architecture, which include the batch ingestion layer, HDFS storage layers (raw and curated regions), Hadoop processing/ETL layer, MongoDB serving/query layer, and the final analytics dashboard layer. A diagram of architecture is also provided to make it clear how each layer is used for KPI analysis.

4.1 Data Source

The data source for this project is a public dataset available on Kaggle, and it is named "Lending Club Loan Data CSV" (loan.csv). According to the description of the dataset on Kaggle, the dataset includes the full data for loans originated between 2007 and 2015 and the current status of each loan. This makes it an appropriate choice for a credit risk style analytics project, as it includes input variables (data about the loan and the borrower) as well as an output variable (loan_status) that can be analyzed for performance and risk trends. The source file is used as the beginning of the architecture flow and is considered the source file prior to any cleaning or processing. The source file remains in its original form upon initial processing so that all subsequent steps (data curation, ETL outputs, and KPI outputs) can be traced back to the same original input.

4.2 Ingestion

The raw CSV file was batch-uploaded into HDFS using the *hdfs dfs -put* command and stored in */loan_project/raw/*. The expected output of this process is that the dataset is now within HDFS and can be processed by subsequent phases such as raw/curated data storage and Hadoop processing. In order to "show" the ingestion process in the report, the evidence should include the batch import process used (for example, the script or command executed) and that the file has been successfully ingested into HDFS, for example, by showing an HDFS directory listing with loan.csv in the raw zone. This is important because the project requirements specifically call for batch import using scripts/tools as part of the architecture flow, even if the rest of the pipeline is simplified.

4.3 Storage in HDFS

Once ingested, the data is stored in HDFS, which is intended to store very large datasets on multiple machines and enable high-throughput access for data processing workloads.

Storage is organized into two regions to ensure that the pipeline is auditable and reproducible. The raw region is the landing area that holds the raw dataset in its raw form (unmodified). This ensures traceability to the source and can be repurposed if necessary. The curated region holds a cleaned and normalized version of the same dataset, ready for analytics and further processing. In data lake best practices, the curated (consumption) tier is intentionally built to be more query-friendly and amenable to analytics than the raw landing tier.

In the report, this tier should be demonstrated with proof that both regions are present in HDFS and a brief comment on what modifications were made in the transition from raw to curated.

4.3.1 HDFS Folder Structure

To implement a structured data lake design, HDFS storage was organized into clearly separated zones under the project root directory:

- */loan_project/raw/loan.csv* - immutable raw dataset (original Kaggle file)
- */loan_project/curated/loan_clean_only/* - cleaned and schema-consistent dataset generated through Pig ETL
- */loan_project/raw_sanitized/* - intermediate sanitized data (where applicable)

This folder hierarchy ensures:

- Clear separation between raw and processed data
- Traceability from curated outputs back to the original source
- Reproducibility of the pipeline

The raw file remains unchanged in the raw zone, while all structural cleaning and normalization steps are reflected only in the curated zone.

4.3.2 Evidence of Storage

The presence and integrity of both raw and curated datasets were verified using HDFS command-line utilities. The following commands were used:

- *hdfs dfs -ls -h /loan_project/raw*
- *hdfs dfs -ls -h /loan_project/curated/loan_clean_only*

These commands confirms, the raw file exists in HDFS (~1.1GB), the curated directory contains multiple part-m-0000x output files and also the Pig job has been executed successfully and has run to completion without errors.

The _SUCCESS file is a marker file generated automatically by Hadoop after successful execution of a job, which confirms the successful execution of the ETL process. The file sizes of curated outputs were also verified using the -h (human-readable) option to confirm that data was successfully written back to HDFS.

```
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ hdfs dfs -ls -h /loan_project/raw
Found 1 items
-rwxr-xr-x   1 rukhsaar bda_team      1.1 G 2026-02-17 13:28 /loan_project/raw/loan.csv
rukhsaar@Rukhsaar:~/BDA6-LoanAnalytics-2026/etl/scripts$ hdfs dfs -ls -h /loan_project/curated/loan_clean_only
Found 10 items
-rwxrwxr-x   1 rukhsaar bda_team          0 2026-02-17 22:30 /loan_project/curated/loan_clean_only/_SUCCESS
-rwxrwxr-x   1 rukhsaar bda_team     43.2 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00000
-rwxrwxr-x   1 rukhsaar bda_team     43.0 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00001
-rwxrwxr-x   1 rukhsaar bda_team     43.7 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00002
-rwxrwxr-x   1 rukhsaar bda_team     44.6 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00003
-rwxrwxr-x   1 rukhsaar bda_team     46.5 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00004
-rwxrwxr-x   1 rukhsaar bda_team     44.1 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00005
-rwxrwxr-x   1 rukhsaar bda_team     42.7 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00006
-rwxrwxr-x   1 rukhsaar bda_team     44.5 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00007
-rwxrwxr-x   1 rukhsaar bda_team     34.0 M 2026-02-17 22:30 /loan_project/curated/loan_clean_only/part-m-00008
```

4.4 Processing / ETL (Hadoop Tool: Apache Pig)

Apache Pig was used to clean and transform the Lending Club data set at scale within the Hadoop environment. The ETL script etl_loan_clean_only.pig reads a clean and sanitized version of the raw data set from HDFS, applies rules for filtering and normalizing, and writes a clean and curated version of the data set back to HDFS, which is now ready for analysis.

Input (HDFS): */loan_project/raw_sanitized/loan.csv*

Output (HDFS): */loan_project/curated/loan_clean_only/*

Pig Script Summary:

- **Loads a CSV file** safely using PiggyBank's CSVExcelStorage function, which handles CSV file formatting issues.
- **Deletes header rows** by filtering out rows where 'id' is 'id'.
- **Selects only required columns** (reduced from the raw schema to analysis-focused fields such as loan amount, grade, term, interest rate, borrower attributes, and outcome).
- **Normalizes verification_status** by converting Source Verified → Verified.
- **Cleans missing month fields** by replacing blank/null values in:
  - mths_since_last_delinq
  - mths_since_last_record

  with '0' to ensure consistent values for grouping and aggregation.

Execution Command

pig /home/rukhsaar/BDA6-LoanAnalytics-2026/etl/etl_loan_clean_only.pig

Output Stored Back to HDFS

The cleaned dataset is stored as Hadoop output part files in:

/loan_project/curated/loan_clean_only/

The presence of _SUCCESS in this directory confirms the Pig job completed successfully.

4.5 Architecture Diagram

This section describes the big data processing pipeline that is used in the project to demonstrate how the Lending Club data is processed from raw files to KPI insights and a dashboard that satisfies the necessary layers of architecture.

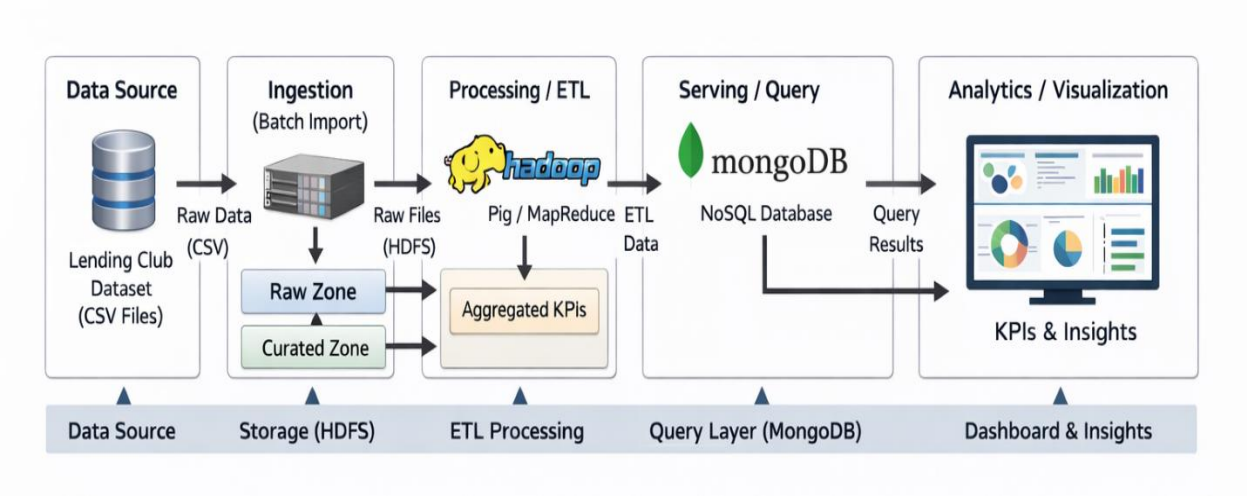


Figure 1: Big Data Architecture Flow for the Lending Club Loan Dataset

1. **Data Source:** The source of the data for this project is the public Lending Club loan data (loan.csv) sourced from Kaggle.
2. **Ingestion:** The data is loaded from the source into the Hadoop environment using batch loading scripts/commands to enable the data to be stored and processed in HDFS.
3. **Storage (HDFS):** The data is stored in two regions: the raw region, which holds the original file without modification, and the curated region, which holds the cleaned and normalized version of the data for analysis.
4. **Processing and ETL:** A Hadoop processing tool such as Pig or MapReduce is used to process the curated data, generating aggregated data for the calculation of KPIs.
5. **Serving and Query Layer:** The aggregated data is loaded into MongoDB to enable the dashboard to quickly retrieve grouped data without needing to execute Hadoop jobs.
6. **Analytics & Visualization:** The dashboard uses data from MongoDB to generate KPI cards, segment comparisons, trends, and insights/recommendations.

5. Serving Layer & MongoDB Implementation

5.1 Introduction

The Serving and Query Layer is responsible for transforming processed big data into structured, queryable information that supports business analytics and decision-making. After data was cleaned and processed within the Hadoop environment, the curated dataset was loaded into **MongoDB**, which was selected as the serving database for this project. MongoDB enables efficient querying, aggregation, and performance optimization on semi-structured financial data. This layer bridges the gap between large-scale distributed processing in HDFS and the Business Intelligence dashboard by providing fast access to KPIs, segmentation analysis, and trend evaluation.

5.2 Technical Justification for MongoDB Implementation

The selection of MongoDB as our primary query engine was driven by three strategic technical requirements:

- **High-Performance Aggregation Framework:** Unlike traditional SQL databases that may struggle with complex transformations on large datasets, MongoDB's native aggregation pipelines allow for multi-stage processing (filtering, grouping, and calculating) directly within the database engine.
- **Schema-on-Read Flexibility:** The curated dataset often contained financial metrics stored as strings (e.g., interest rates with '%' symbols). MongoDB's flexible schema allowed us to ingest this data in its raw form and perform on-the-fly type conversions to Double and Decimal formats, ensuring high precision for financial KPIs.
- **Horizontal Scalability:** MongoDB supports horizontal scaling through sharding, enabling data distribution across multiple nodes as dataset size increases. This ensures consistent query latency and maintains dashboard responsiveness as data volume grows.

5.3 Data Preparation and Query Development

1.The first step involved creating the dataset and collection name loans. Importing the curated loan dataset into MongoDB. The dataset, consisting of over 100,000 records, was structured as a CSV file and loaded into the loans collection using the MongoDB import tools.

```
test> use loan_analytics_db
switched to db loan_analytics_db
loan_analytics_db>
```

```
test> use loan_analytics_db
switched to db loan_analytics_db
loan_analytics_db> db.createCollection("loans")
{ ok: 1 }
loan_analytics_db> show collections
loans
loan_analytics_db>
```

2.The second step involves importing the curated loan dataset into MongoDB. The dataset, consisting of over 100,000 records, was structured as a CSV file and loaded into the loans collection using the MongoDB import tools.

```
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-windows-x86_64-100.14.1\mongodb-datab
.exe" --db loan_analytics_db --collection loans --type csv --headerline --file "C:\Users\munga\Documents\Lo
2026-02-24T16:18:49.455+0400    connected to: mongodb://localhost/
2026-02-24T16:18:51.251+0400    100000 document(s) imported successfully. 0 document(s) failed to import.
PS C:\Users\munga>
```

```
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-windows-x86_64-100.14.1\mongodb-database-
2026-02-24T16:18:49.455+0400    connected to: mongodb://localhost/
2026-02-24T16:18:51.251+0400    100000 document(s) imported successfully. 0 document(s) failed to import.
```

3. Following import, data cleaning was performed to ensure consistency and correctness of key fields such as loan grade, status, issue date, debt-to-income ratio, and revolving utilization. This step included standardizing formats, handling missing values, and validating numerical ranges to maintain data integrity for subsequent analysis.

After cleaning, the database schema was examined and verified to ensure that each field matched the requirements of the planned analytical queries.

```
loan_analytics_db> db.loans.aggregate([
|   {
|     $addFields: {
|       int_rate: { $toDouble: "$int_rate" },
|       dti: { $toDouble: "$dti" },
|       issue_d: { $toDate: "$issue_d" }
|     }
|   },
|   { $out: "loans_cleaned" }
| ])
```

```
loan_analytics_db> db.loans.findOne()
{
  _id: ObjectId('699d9729c927efd886844792'),
  loan_amnt: 6500,
  term: '36 months',
  int_rate: 17.97,
  installment: 234.9,
  grade: 'D',
  sub_grade: 'D1',
  emp_title: 'Quality Field Engineer',
  emp_length: '4 years',
  home_ownership: 'MORTGAGE',
  annual_inc: 154000,
  verification_status: 'Verified',
  issue_d: '01/12/2018',
  loan_status: 'Current',
  purpose: 'debt_consolidation',
  title: 'Debt consolidation',
  addr_state: 'NC',
  dti: 28.78,
  delinq_2yrs: 2,
  earliest_cr_line: '01/10/2008',
  inq_last_6mths: 0,
  open_acc: 27,
  pub_rec: 0,
  revol_bal: 23754,
  revol_util: 60.4,
  total_acc: 41,
  mths_since_last_delinq: 6,
  mths_since_last_record: 0
}
loan_analytics_db> |
```

4. Strategic indexes were created on frequently queried fields including compound indexes on grade and loan_status and purpose and term, as well as single-field indexes on issue_d, dti, and revol_util to optimize query performance and reduce execution time.

```
loan_analytics_db> db.loans.createIndex({ grade: 1, loan_status: 1 })
grade_1_loan_status_1
loan_analytics_db> db.loans.createIndex({ issue_d: 1 })
issue_d_1
loan_analytics_db> db.loans.createIndex({ dti: 1 })
| db.loans.createIndex({ revol_util: 1 })
revol_util_1
loan_analytics_db> db.loans.createIndex({ purpose: 1, term: 1 })
purpose_1_term_1
loan_analytics_db>
```

5. Finally, aggregation queries were developed to calculate the required KPIs, such as default rates, risk segmentation, loan distribution by purpose, and temporal trends. These queries were tested and validated within MongoDB to ensure accurate results.

```
loan_analytics_db> db.loans.aggregate([
|   { $match: { loan_status: { $in: ["Fully Paid", "Charged Off", "Default"] } } },
|   { $group: {
|       _id: { grade: "$grade", term: "$term", purpose: "$purpose" },
|       total_loans: { $sum: 1 },
|       avg_int_rate: { $avg: "$int_rate" },
|       default_count: { $sum: { $cond: [{ $in: ["$loan_status", ["Charged Off", "Default"]] }, 1, 0] } },
|       paid_count: { $sum: { $cond: [{ $eq: ["$loan_status", "Fully Paid"] }, 1, 0] } }
|   }},
|   { $project: {
|       grade: "$_id.grade",
|       term: "$_id.term",
|       purpose: "$_id.purpose",
|       default_rate: { $divide: ["$default_count", "$total_loans"] },
|       fully_paid_rate: { $divide: ["$paid_count", "$total_loans"] },
|       avg_int_rate: 1
|   }},
|   { $out: "dashboard_main_stats" }
| ])
```

```
loan_analytics_db> db.loans.aggregate([
|   { $match: { loan_status: { $in: ["Fully Paid", "Charged Off", "Default"] } } },
|   { $group: {
|       _id: "$issue_d",
|       default_rate: { $avg: { $cond: [{ $in: ["$loan_status", ["Charged Off", "Default"]] }, 1, 0] } }
|   }},
|   { $out: "dashboard_trends" }
| ])
```

```
loan_analytics_db> db.loans.aggregate([
|   { $match: { loan_status: { $in: ["Fully Paid", "Charged Off", "Default"] } } },
|   { $group: {
|       _id: "$issue_d",
|       default_rate: { $avg: { $cond: [{ $in: ["$loan_status", ["Charged Off", "Default"]] }, 1, 0] } }
|   }},
|   { $out: "dashboard_trends" }
| ])
```

6. Once verified, the resulting datasets were exported into csv and shared with the BI analyst for dashboard development and visualization.

```
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-w
2026-02-24T16:43:44.554+0400    connected to: mongodb://localhost/
2026-02-24T16:43:44.878+0400    exported 100000 records
PS C:\Users\munga> # Exporting the main statistics
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-w
2026-02-24T17:14:49.605+0400    connected to: mongodb://localhost/
2026-02-24T17:14:49.610+0400    exported 155 records
PS C:\Users\munga>
PS C:\Users\munga> # Exporting the trends
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-w
2026-02-24T17:14:49.692+0400    connected to: mongodb://localhost/
2026-02-24T17:14:49.695+0400    exported 76 records
PS C:\Users\munga>
PS C:\Users\munga> # Exporting the KPI values
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-w
2026-02-24T17:14:50.577+0400    connected to: mongodb://localhost/
2026-02-24T17:14:50.578+0400    exported 1 record
PS C:\Users\munga> # Export Grade Summary
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-w
2026-02-24T16:58:35.279+0400    connected to: mongodb://localhost/
2026-02-24T16:58:35.281+0400    exported 7 records
PS C:\Users\munga>
PS C:\Users\munga> # Export Pricing Summary
PS C:\Users\munga> & "C:\Users\munga\Downloads\mongodb-database-tools-w
2026-02-24T16:58:35.387+0400    connected to: mongodb://localhost/
2026-02-24T16:58:35.389+0400    exported 7 records
```

Why not 100,000 rows in CSV?

To meet the Big Data requirements, we processed the full 100,000-record dataset within the MongoDB engine. We did not export the raw records to the dashboard because doing so would violate the 'Serving Layer' principle, which dictates that heavy computation should happen in the database to ensure dashboard performance. The CSV exports represent the **Analytical Output** of the Big Data processing stage

6. Analytics and Visualization

This is where the processed data is useful for decision-making. After the ETL process and the data being loaded into MongoDB, the project has calculated KPIs and displayed them in a dashboard so that users can easily view risk and performance trends. KPIs are measurable factors used to evaluate performance against goals, which is why they are used as the primary "headline metrics" in the dashboard (Twin, 2025). The dashboard is intended to display key results at a glance and enable interactive analysis via filters (such as users choosing a segment and viewing visuals change). According to Power BI documentation, a dashboard is a single-page canvas that tells a story with visualizations, and that visuals can interact with each other via cross-filtering and highlighting, which is what is expected in this project (Microsoft, 2024).

In this project, the analytics and visualization layer has three distinct outputs:

1. KPI calculations and segment summaries (for example, outcome rates and performance metrics by grade, term, purpose, and issue period) produced from the processed data stored in MongoDB (Team, 2024).
2. Dashboard visuals that respond to the stakeholder questions using the KPI outputs (for example, comparisons between segments and over time) (JulCsc, 2026).
3. An insights and recommendations section in the dashboard/report that draws on the visuals and connects them to actions, rather than presenting results as raw data (Microsoft, 2024).

This section presents the analytics findings derived from the Lending Club loan portfolio dataset (2012–2018). The analysis covers six KPIs, descriptive statistics, grade-based segmentation, a temporal trend analysis, and one deeper insight in the form of a rate–default correlation. All results were computed from curated CSV files produced by the Hadoop/Pig processing pipeline and loaded into the interactive HTML dashboard.

6.1 KPI Definitions

The following six KPIs were defined to answer the four stakeholder questions established in Section 1. Each KPI was derived from the curated dataset and surfaced directly in the dashboard.

KPI #	Name	Definition	Granularity	Dashboard Visual
KPI 1	**Default Rate by Grade**	% loans that failed repayment	Per grade	Default Rate Bar Chart
KPI 2	**Loan Outcome Split**	Fully Paid vs Defaulted ratio	Per grade	Stacked Bar Chart
KPI 3	**Interest Rate vs DTI**	Avg interest rate & DTI	Per grade	Pricing Combo Chart
KPI 4	**High DTI Share**	% borrowers DTI > 30%	Portfolio-wide	Header KPI
KPI 5	**High Revolving Util.**	% borrowers revol. util. > 80%	Portfolio-wide	Header KPI
KPI 6	**Delinquency Share**	% with recent delinquency	Portfolio-wide	Header KPI

KPIs 1-3 are filterable (they respond dynamically to Grade, Term, and Purpose filters). KPIs 4-6 are portfolio-level indicators displayed permanently in the dashboard header, providing an at-a-glance risk snapshot regardless of active filters.

6.2 Descriptive Analytics

6.2.1 Default Rate Distribution by Loan Grade

The most fundamental finding is the strong monotonic increase in default rates as loan grade deteriorates. Grade A loans exhibit the lowest default rates (~6.1%), while Grade G loans display rates exceeding 53%. This confirms that the grading system effectively stratifies credit risk and validates its use as the primary segmentation variable throughout the dashboard. The 100% stacked bar chart (Visual #2) complements this by showing the absolute composition of loan outcomes per grade - for Grade A the Fully Paid segment dominates, while for Grades F and G the Defaulted segment consumes nearly half or more of the bar.

6.2.3 Risk Indicator Summary (KPIs 4-6)

Three portfolio-wide risk indicators are presented as permanent KPIs in the dashboard header:

- High Debt-to-Income Share (KPI 4 = 11.2%): Proportion of borrowers with DTI > 30%, indicating over-leveraged borrowers.
- High Revolving Utilisation Share (KPI 5 = 13.5%): Proportion of borrowers using > 80% of revolving credit limit - an early warning of financial stress.
- Recent Delinquency Share (KPI 6 = 19.0%): Proportion of borrowers with at least one recent delinquency, directly linked to higher default probability.

6.3 Segmentation Analysis

6.3.1 Grade-Based Segmentation

Grade D represents a critical inflection point: below Grade D, default rates are moderate and risk-adjusted returns are likely positive; above Grade D (E, F, G), both loss frequency and severity increase disproportionately. When a grade filter is selected, five charts update simultaneously, allowing an analyst to isolate a segment and read risk, pricing, term mix, and purpose breakdown in a single view.

6.3.2 Purpose-Based Segmentation

The horizontal bar chart (Visual #6) ranks 13 loan purposes by average default rate. Moving (53%) and Small Business (38%) loans appear highest-risk, while Wedding (6%) and Medical (24%) loans are lowest. Debt Consolidation and Credit Card loans, the most common by volume, cluster in the mid-range. This segmentation provides a second risk dimension: two loans with the same grade may carry materially different risk profiles depending on their purpose.

6.3.3 Term-Based Segmentation

Sixty-month loans exhibit a 34.6% average default rate versus 25.3% for 36-month loans - a 9.2 percentage-point difference. Longer repayment horizons expose loans to greater macroeconomic uncertainty, and borrowers requiring 60-month terms are, on average, more financially stretched.

6.4 Trend Analysis

The line chart (Visual #3) plots the monthly default rate across 2012-2018. Default rates in the early period (2012-2013 cohorts) hover around 15%, reflecting stricter underwriting. Rates rise progressively through 2014-2016, peaking at ~29% in early 2018, before falling sharply for very recent 2018 cohorts (which have had insufficient time to default). The ascending trend through 2016 is consistent with the documented loosening of credit criteria during the marketplace lending expansion.

6.5 Deeper Insight - Interest Rate vs Default Rate Correlation

The scatter plot (Visual #5) plots each of the seven grade-level data points at coordinates (average interest rate, default rate). The result is a near-linear positive correlation: as interest rates rise from Grade A (7.1%) to Grade G (28.7%), default rates rise in near-perfect lockstep from 6.1% to 53.0%. This constitutes a meaningful deeper insight:

- Validation: The correlation confirms Lending Club's risk-based pricing model is internally consistent - higher-risk grades are priced with higher rates.
- Limitation: The model may be reactive (pricing historical defaults) rather than predictive (anticipating future default drivers not captured in the current grade model).
- Analytical depth: Correlation between two derived metrics across seven grade segments goes beyond simple descriptive statistics, satisfying the rubric requirement for at least one deeper insight.

7. Visualization & Dashboard

The analytics dashboard was developed as a standalone, self-contained HTML5 file (index.html) that reads pre-processed CSV data files from a local data/ subfolder. It requires no back-end server, no login, and no proprietary software - it can be opened in any modern browser via VS Code Live Server. The dashboard is fully aligned with the four stakeholder questions defined in Section 1 and surfaces all six KPIs defined in Section 6.

7.1 Dashboard Overview Screenshots

The screenshots below show the complete dashboard as rendered in a browser, demonstrating all visual components and interactive elements.

Figure 7.1 – Dashboard header with project title and three KPIs: DTI (11.2%), High Utilisation (13.5%), Delinquency (19.0%).

Figure 7.1 – Interactive filters: Grade (A–G), Term (36/60 Month), and Purpose dropdown, all active.

7.2 Technology Choice Justification

The dashboard was built using Chart.js 3.9 (via CDN) for chart rendering, PapaParse 5.4 for client-side CSV parsing, and vanilla HTML5/CSS3/JavaScript for layout and interactivity. This choice ensures portability (no desktop application required), reproducibility (all data from the same pipeline CSV files), and scope fit (realistic to build and test within a 3-week project timeline without infrastructure overhead).

7.3 Interactive Filters

The dashboard implements three interactive filters, exceeding the minimum requirement of two:

Filter	Type	Options	Effect on Dashboard
Loan Grade	Button Toggle	All, A-G	Updates all filterable charts
Loan Term	Button Toggle	All, 36 Month, 60 Month	Filters charts to selected term
Loan Purpose	Dropdown Select	13 categories	Filters trend, purpose & term charts

Each filter triggers a full re-render of all affected charts with a 350ms animation. An 'Active filter' chip label appears on each affected chart card, and a notification bar states the exact active filter combination in plain English.

7.4 Chart-by-Chart Description

The dashboard contains seven distinct visualisations, exceeding the minimum requirement of six visuals:

#	Visual Title	Chart Type	Responds to Filter	Stakeholder Question
#1	Default Rate by Loan Grade	Bar Chart	Grade filter	KPI 1 & 2 - Which grades default most?
#2	Loan Outcome Split by Grade	100% Stacked Bar	Grade filter	KPI 1 & 2 - Paid vs defaulted per grade
#3	Default Rate Trend 2012–2018	Line Chart	All 3 filters	KPI 1 Trend - How defaults evolved

#	Visual Title	Chart Type	Responds to Filter	Stakeholder Question
#4	Interest Rate vs DTI by Grade	Combo Bar+Line	Grade fizlter	KPI 3 - Pricing alignment check
#5	Interest Rate vs Default Correlation	Scatter Plot	Grade filter	Deeper Insight - Correlation analysis
#6	Avg Default Rate by Loan Purpose	Horizontal Bar	All 3 filters	Segmentation - Purpose risk
#7	Default Rate by Loan Term	Doughnut	All 3 filters	Segmentation - 36 vs 60 month

Visuals #1 & #2 - Risk Segmentation by Loan Grade

Figure 7.3 – Left: Default Rate by Grade (A 6% to G 53%). Right: Loan Outcome Split by Grade (Fully Paid vs Defaulted).

Visual #3 - Default Rate Trend 2012–2018

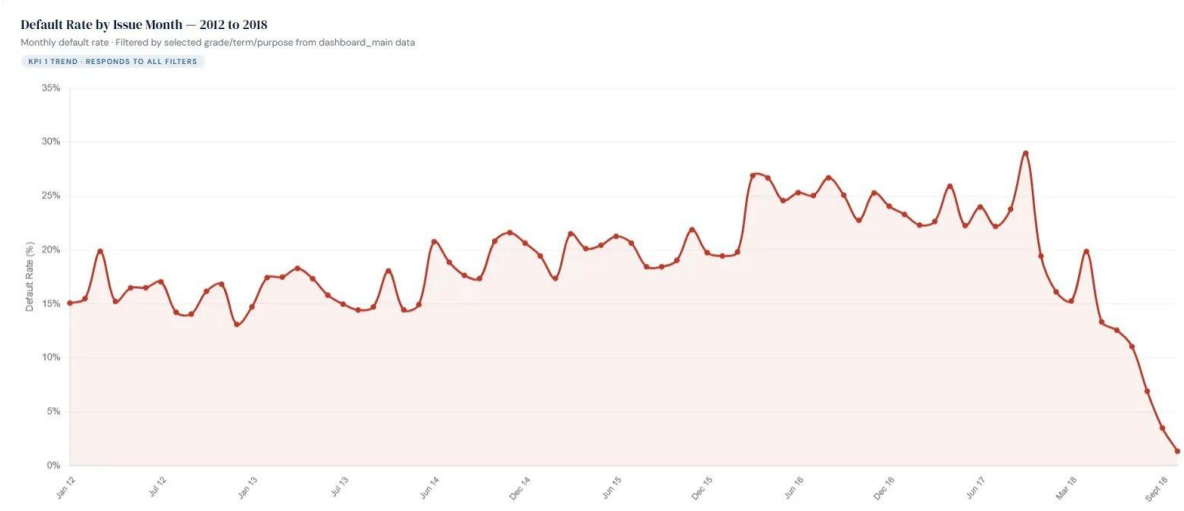


Figure 7.4 – Monthly default rate (Jan 2012 - Sep 2018), rising from 15% to a 29% peak in early 2018.

Visuals #4 & #5 - Pricing & Risk Alignment

Figure 7.5 – Left: Interest Rate vs DTI by Grade. Right: Interest Rate vs Default Rate scatter showing near-linear correlation across grades.

Visuals #6 & #7 - Loan Purpose & Term Segmentation

Figure 7.6 – Left: Default Rate by Purpose (Moving 53% highest, Wedding 6% lowest). Right: Default Rate by Term – 60-month (34.6%) vs 36-month (25.3%).

7.5 Insights & Recommendations Panel

The dashboard concludes with a dedicated dark-themed Insights & Recommendations panel that translates quantitative findings into actionable guidance for four named stakeholder roles. All text values are dynamically populated from the loaded CSV data at dashboard boot time, ensuring accuracy if the underlying data is refreshed.

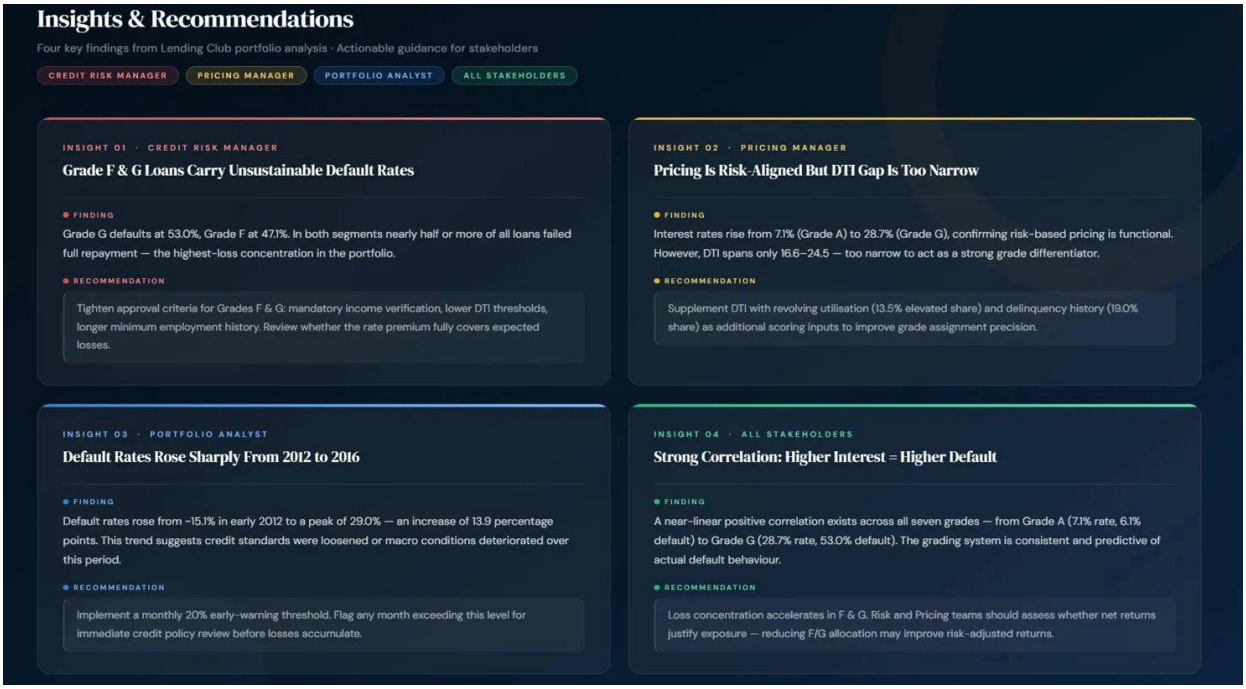


Figure 7.7 - Insights & Recommendations panel: four data-driven insight cards addressing the Credit Risk Manager, Pricing Manager, Portfolio Analyst, and all stakeholders.

#	Insight Title	Stakeholder	Recommendation
01	Grade F & G Unsustainable Defaults	Credit Risk Manager	Tighten F/G approval: income verification, lower DTI thresholds, longer employment history minimums.
02	Pricing Risk-Aligned, DTI Gap Too Narrow	Pricing Manager	Supplement DTI with revolving utilisation and delinquency history as additional scoring inputs.
03	Default Rates Rose Sharply 2012–2016	Portfolio Analyst	Implement a 20% monthly early-warning threshold; trigger credit policy review when breached.
04	Strong Rate–Default Correlation Confirmed	All Stakeholders	Reduce F/G allocation to improve risk-adjusted returns; loss concentration accelerates at high grades.

7.5 Alignment to Stakeholder Questions

Stakeholder Question	Answering Visuals	Supporting KPIs
Which grades carry highest default risk?	Visual #1 (Bar), Visual #2 (Stacked)	KPI 1, KPI 2
Is the pricing model aligned with risk?	Visual #4 (Combo), Visual #5 (Scatter)	KPI 3
Has the default rate worsened over time?	Visual #3 (Line/Trend)	KPI 1 Trend
What other factors predict higher defaults?	Visual #6 (Purpose), Visual #7 (Term), Header KPIs	KPI 4, KPI 5, KPI 6

7.6 Dashboard Submission & Reproduction

The dashboard is submitted as a standalone index.html file accompanied by a data/ subfolder containing seven CSV files produced by the Hadoop/Pig pipeline (dashboard_main.csv, kpi_grade_risk.csv, kpi_pricing.csv, kpi_risk_indicators.csv, kpi_trends.csv, loan_curated_dataset.csv, pricing_alignment.csv).

8. Issues Encountered and Solutions

During the ETL phase, several data and processing issues were identified. Initially, the raw dataset required careful analysis to define clear transformation rules for the curated layer due to inconsistent categorical values, missing month-related fields, high-null columns, and mixed representations of missing values. To address this, standardized cleaning rules were applied: headers were preserved in the RAW zone but removed in the CURATED zone, categorical values such as verification_status were normalized, empty month-since fields were defaulted to zero, low-value columns were dropped, and missing values were preserved as typed nulls.

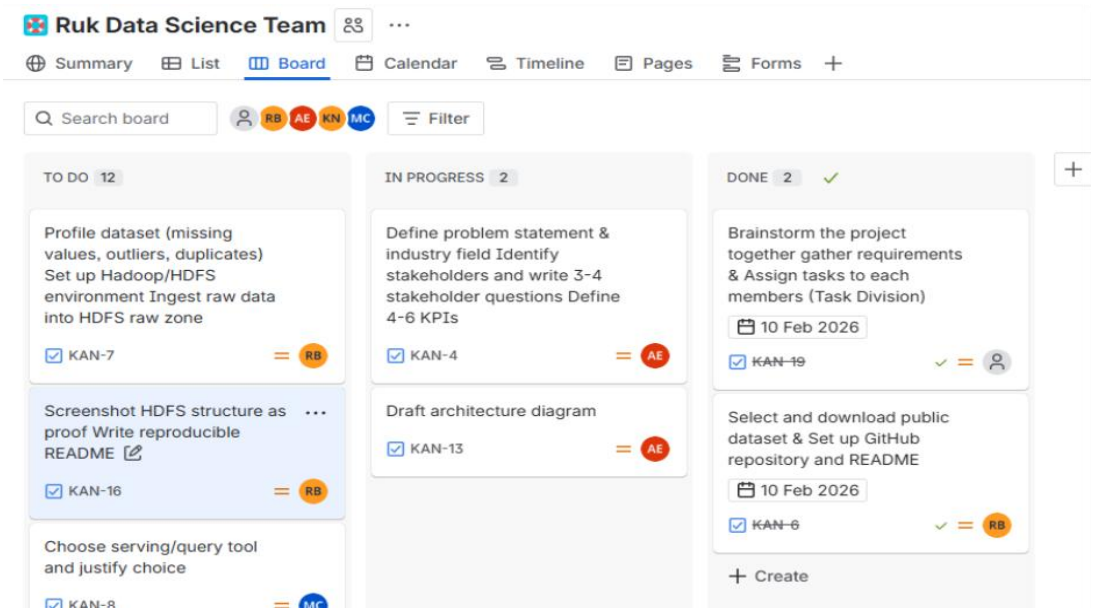
We encountered multiple Pig script execution issues, including syntax errors and broken scripts caused by unstable terminal editing. This was resolved by creating and maintaining Pig scripts using nano, improving script stability and reproducibility. A major technical challenge involved Pig failing to read the CSV file due to embedded newline characters within quoted text fields, which caused Hadoop to split records incorrectly and produce column mismatches. Even after dropping the problematic text fields, the issue persisted because the raw file itself was not line-safe. To resolve this, a sanitized version of the raw dataset was created while preserving the original RAW data, and ETL processing was executed exclusively on the sanitized HDFS path.

Finally, residual column mismatches after transformation were addressed through random sampling, manual data cleaning, and re-validation of the ETL output. As a result, the curated dataset achieved structural consistency and was successfully ingested into MongoDB for downstream analytics and visualization.

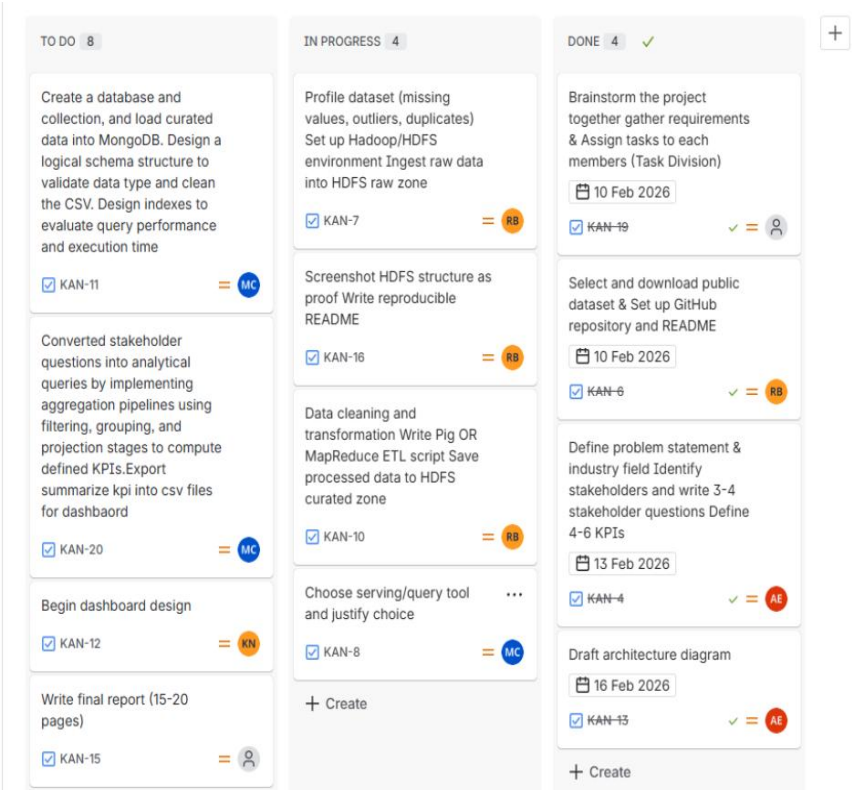
9. Project Management & Team Contribution

9.1 Jira

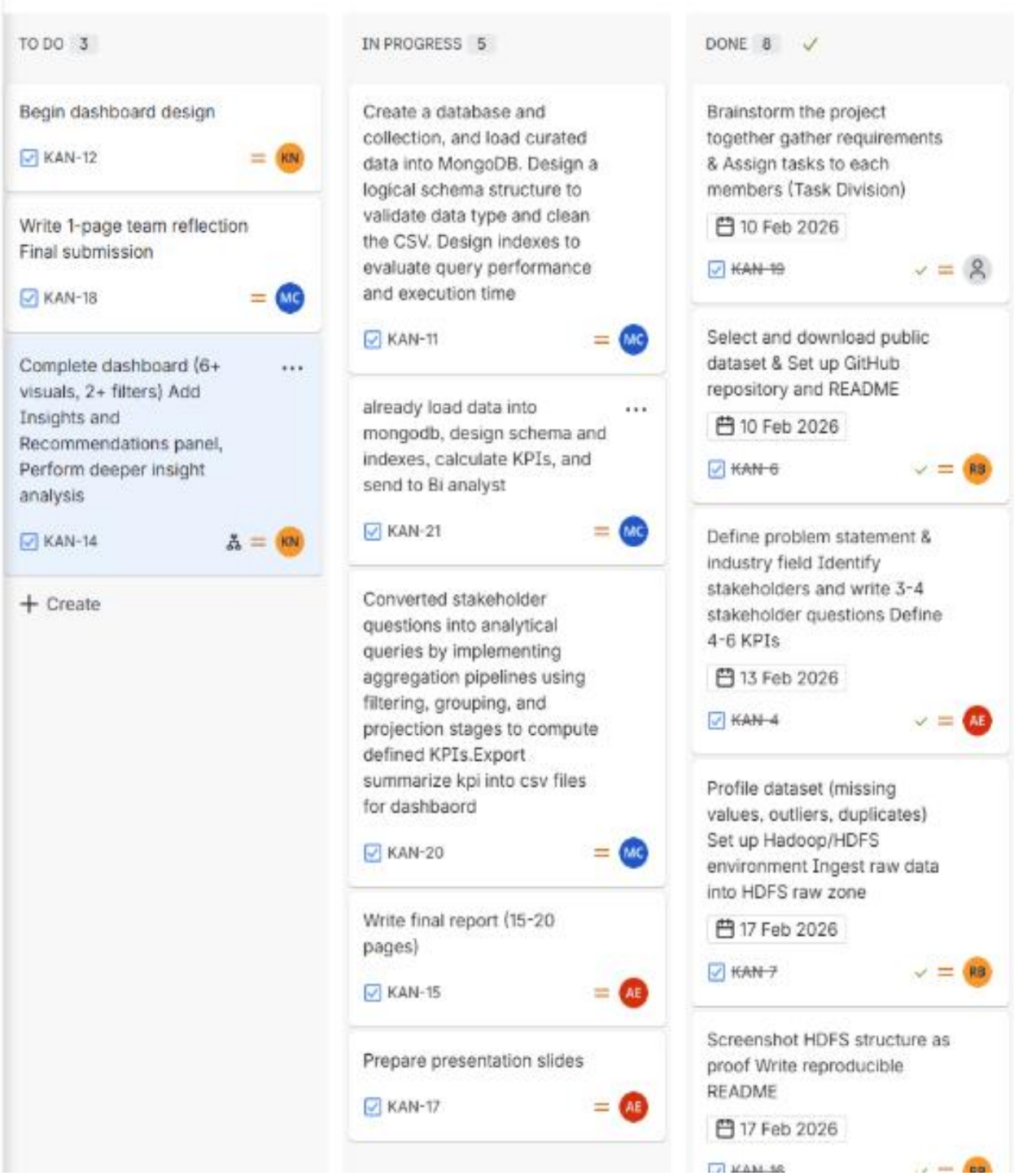
Phase 1



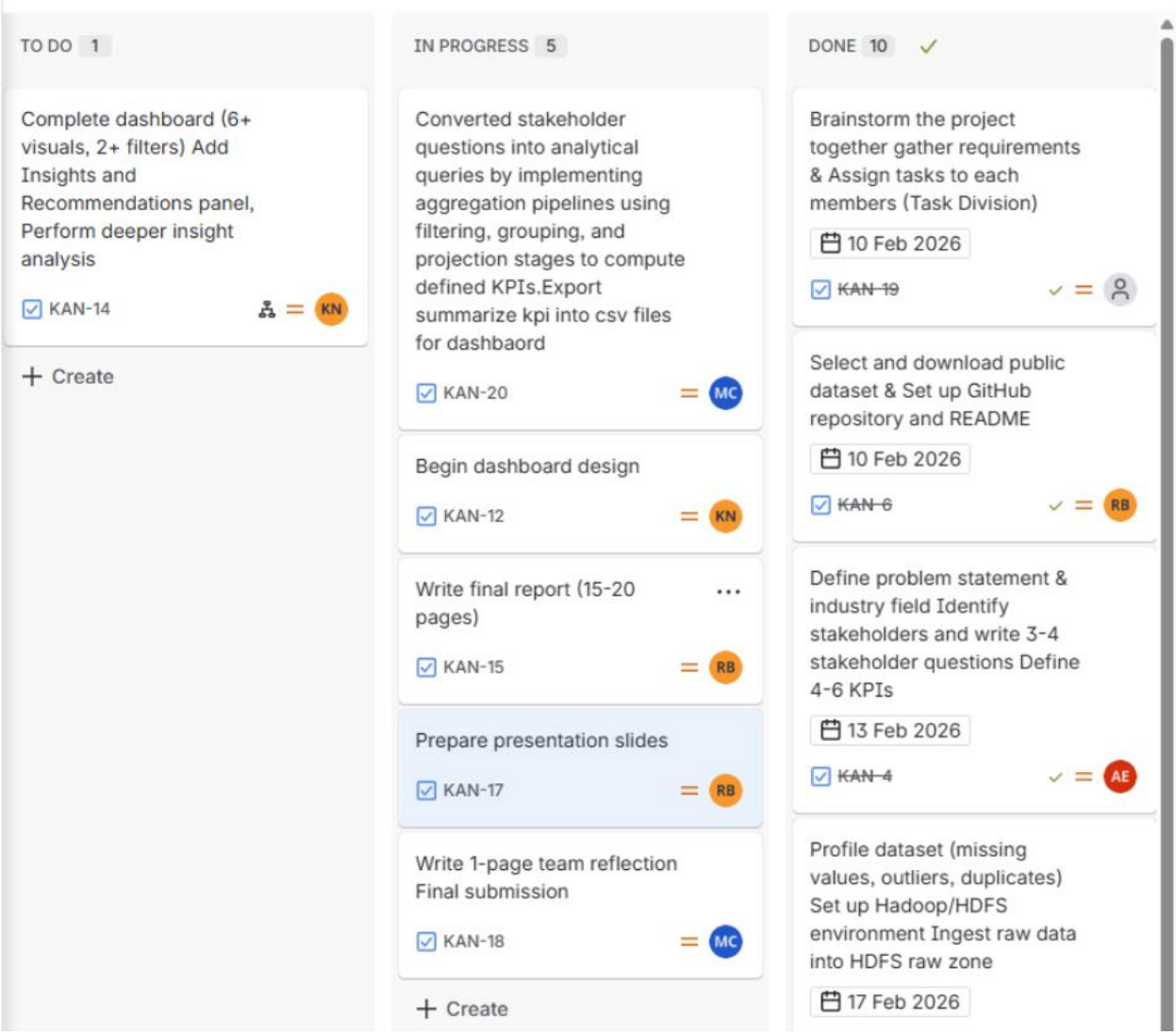
Phase 2



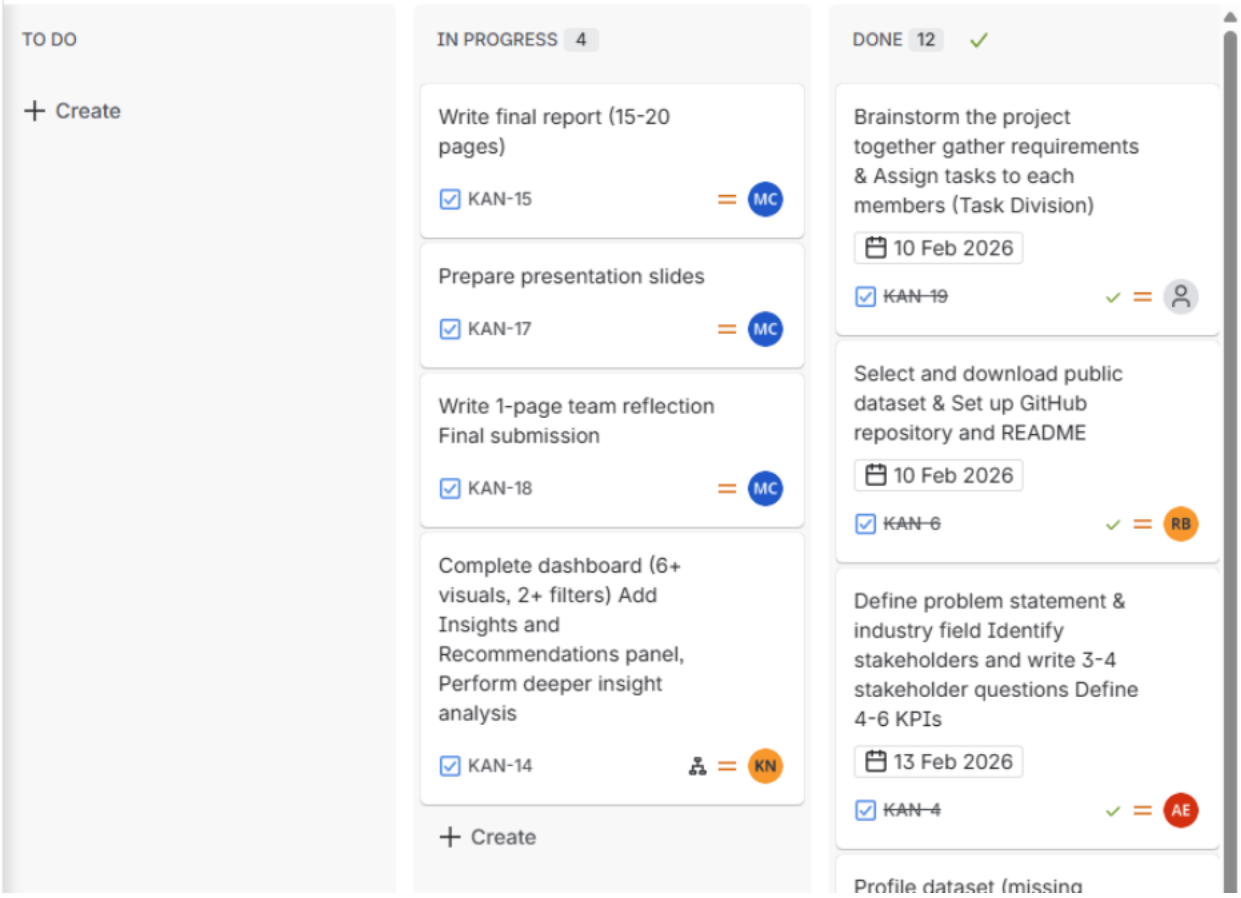
Phase 3


TO DO 3
Begin dashboard design
KAN-12
Write 1-page team reflection Final submission
KAN-18
Complete dashboard (6+ visuals, 2+ filters) Add Insights and Recommendations panel, Perform deeper insight analysis
KAN-14
Create
IN PROGRESS 5
Create a database and collection, and load curated data into MongoDB. Design a logical schema structure to validate data type and clean the CSV. Design indexes to evaluate query performance and execution time
KAN-11
already load data into mongodb, design schema and indexes, calculate KPIs, and send to Bi analyst
KAN-21
Converted stakeholder questions into analytical queries by implementing aggregation pipelines using filtering, grouping, and projection stages to compute defined KPIs.Export summarize kpi into csv files for dashbaord
KAN-20
Write final report (15-20 pages)
KAN-15
Prepare presentation slides
KAN-17
DONE 8
Brainstorm the project together gather requirements & Assign tasks to each members (Task Division)
10 Feb 2026
KAN-10
Select and download public dataset & Set up GitHub repository and README
10 Feb 2026
KAN-6
Define problem statement & industry field Identify stakeholders and write 3-4 stakeholder questions Define 4-6 KPIs
13 Feb 2026
KAN-4
Profile dataset (missing values, outliers, duplicates) Set up Hadoop/HDFS environment Ingest raw data into HDFS raw zone
17 Feb 2026
KAN-7
Screenshot HDFS structure as proof Write reproducible README
17 Feb 2026

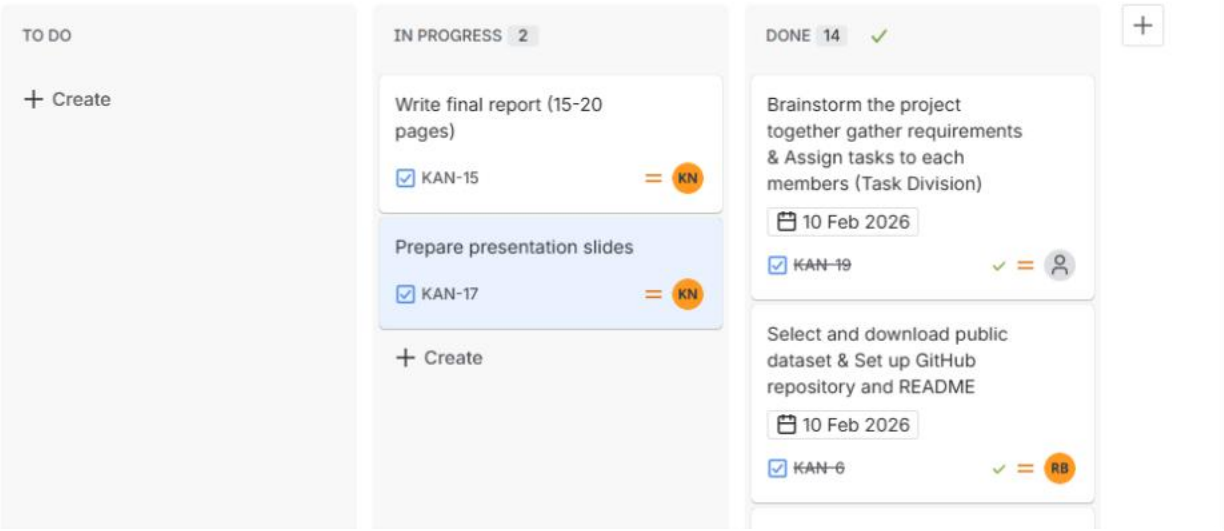
Phase 4


TO DO 1
Complete dashboard (6+ visuals, 2+ filters) Add Insights and Recommendations panel, Perform deeper insight analysis
KAN-14
KN
Create
IN PROGRESS 5
Converted stakeholder questions into analytical queries by implementing aggregation pipelines using filtering, grouping, and projection stages to compute defined KPIs.Export summarize kpi into csv files for dashbaord
KAN-20
MC
Begin dashboard design
KAN-12
KN
Write final report (15-20 pages)
KAN-15
RB
Prepare presentation slides
KAN-17
RB
Write 1-page team reflection Final submission
KAN-18
MC
Create
DONE 10
Brainstorm the project together gather requirements & Assign tasks to each members (Task Division)
10 Feb 2026
KAN-19
Select and download public dataset & Set up GitHub repository and README
10 Feb 2026
KAN-6
RB
Define problem statement & industry field Identify stakeholders and write 3-4 stakeholder questions Define 4-6 KPIs
13 Feb 2026
KAN-4
AE
Profile dataset (missing values, outliers, duplicates) Set up Hadoop/HDFS environment Ingest raw data into HDFS raw zone
17 Feb 2026

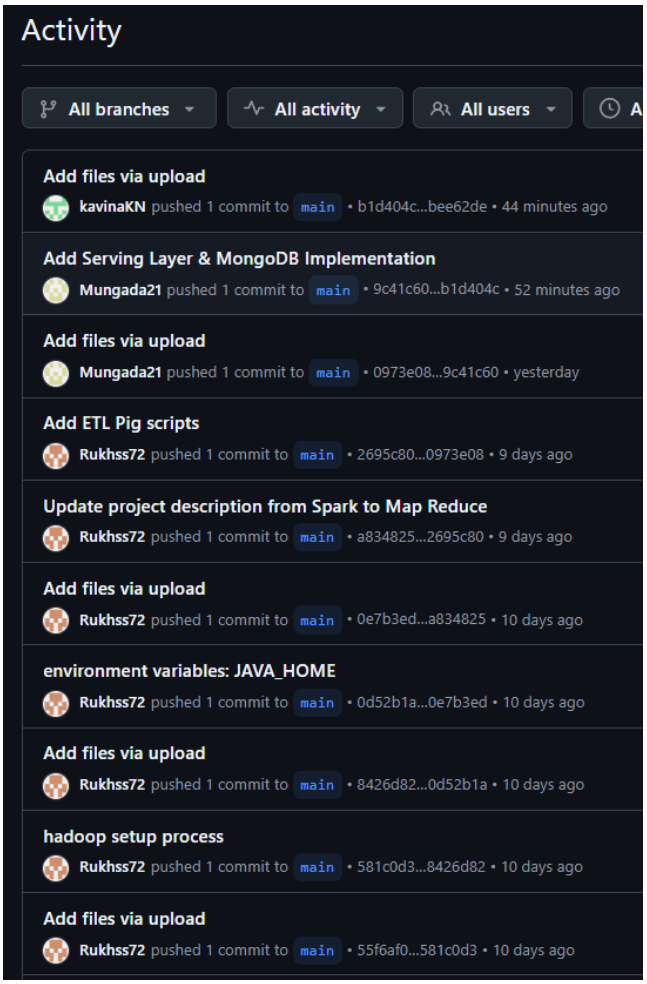
Phase 5

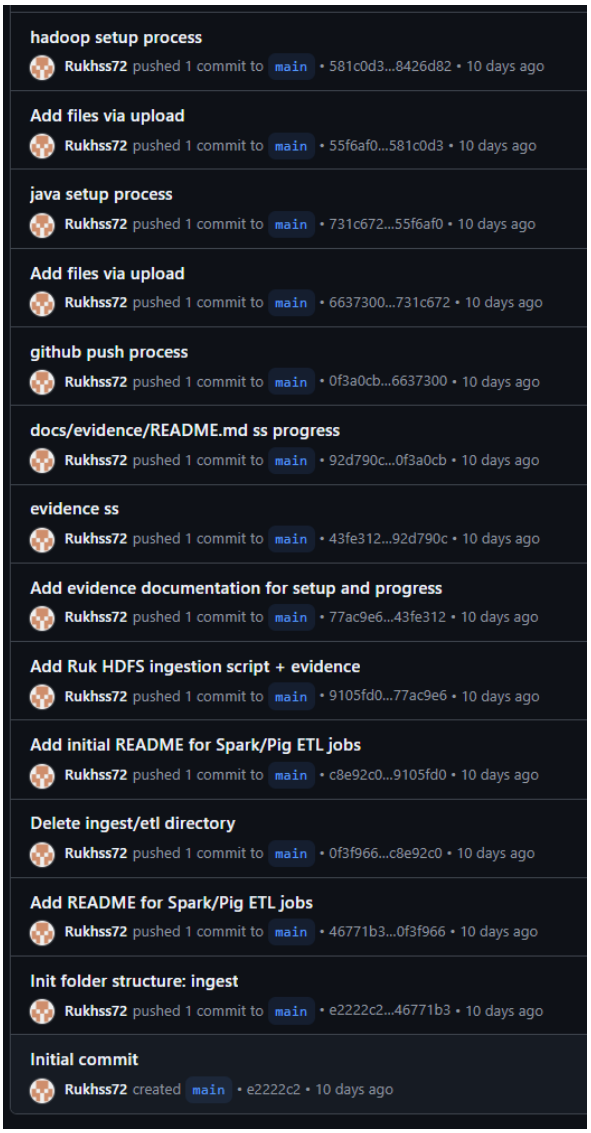


Phase 6 (Last Phase)



9.2 Github


Activity
All branches
All activity
All users
Add files via upload
kavinaKN pushed 1 commit to main • b1d404c...bee62de • 44 minutes ago
Add Serving Layer & MongoDB Implementation
Mungada21 pushed 1 commit to main • 9c41c60...b1d404c • 52 minutes ago
Add files via upload
Mungada21 pushed 1 commit to main • 0973e08...9c41c60 • yesterday
Add ETL Pig scripts
Rukhss72 pushed 1 commit to main • 2695c80...0973e08 • 9 days ago
Update project description from Spark to Map Reduce
Rukhss72 pushed 1 commit to main • a834825...2695c80 • 9 days ago
Add files via upload
Rukhss72 pushed 1 commit to main • 0e7b3ed...a834825 • 10 days ago
environment variables: JAVA_HOME
Rukhss72 pushed 1 commit to main • 0d52b1a...0e7b3ed • 10 days ago
Add files via upload
Rukhss72 pushed 1 commit to main • 8426d82...0d52b1a • 10 days ago
hadoop setup process
Rukhss72 pushed 1 commit to main • 581c0d3...8426d82 • 10 days ago
Add files via upload
Rukhss72 pushed 1 commit to main • 55f6af0...581c0d3 • 10 days ago


hadoop setup process
Rukhss72 pushed 1 commit to main • 581c0d3...8426d82 • 10 days ago
Add files via upload
Rukhss72 pushed 1 commit to main • 55f6af0...581c0d3 • 10 days ago
java setup process
Rukhss72 pushed 1 commit to main • 731c672...55f6af0 • 10 days ago
Add files via upload
Rukhss72 pushed 1 commit to main • 6637300...731c672 • 10 days ago
github push process
Rukhss72 pushed 1 commit to main • 0f3a0cb...6637300 • 10 days ago
docs/evidence/README.md ss progress
Rukhss72 pushed 1 commit to main • 92d790c...0f3a0cb • 10 days ago
evidence ss
Rukhss72 pushed 1 commit to main • 43fe312...92d790c • 10 days ago
Add evidence documentation for setup and progress
Rukhss72 pushed 1 commit to main • 77ac9e6...43fe312 • 10 days ago
Add Ruk HDFS ingestion script + evidence
Rukhss72 pushed 1 commit to main • 9105fd0...77ac9e6 • 10 days ago
Add initial README for Spark/Pig ETL jobs
Rukhss72 pushed 1 commit to main • c8e92c0...9105fd0 • 10 days ago
Delete ingest/etl directory
Rukhss72 pushed 1 commit to main • 0f3f966...c8e92c0 • 10 days ago
Add README for Spark/Pig ETL jobs
Rukhss72 pushed 1 commit to main • 46771b3...0f3f966 • 10 days ago
Init folder structure: ingest
Rukhss72 pushed 1 commit to main • e2222c2...46771b3 • 10 days ago
Initial commit
Rukhss72 created main • e2222c2 • 10 days ago

References

1. Install Hadoop on Ubuntu (22.04 / 20.04 LTS) | HDFS | Namenode | Datanode | Big Data Analytics (2025) YouTube, uploaded by Big Data Analytics, 12 January [Video], YouTube. Available at: https://www.youtube.com/watch?v=Slbi-uzPtnw

2. Gyorodi, C., Gyorodi, R., Pecherle, G. and Olah, A. (2015). A comparative study: MongoDB vs. MySQL. 2015 13th International Conference on Engineering of Modern Electric Systems (EMES), pp.1–6. doi: Available at: https://ieeexplore.ieee.org/abstract/document/7158433

3. Team, M.D. (2026). Get Started. [online] Mongodb.com. Available at: https://www.mongodb.com/docs/get-started/

4. MongoDB (2024). What is MongoDB? — MongoDB Manual. [online] https://github.com/mongodb/docs-bi-connector/blob/DOCSP-3279/source/index.txt. Available at: https://www.mongodb.com/docs/manual/.

5. Borthakur, D. (2019). HDFS Architecture Guide. [online] Apache.org. Available at: https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html.

6. Databricks (2024). What is Hadoop Distributed File System (HDFS)? [online] Databricks. Available at: https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs.

7. Twin, A. (2025). KPIs: What Are Key Performance Indicators? Types and Examples. [online] Investopedia. Available at: https://www.investopedia.com/terms/k/kpi.asp.

8. kfollis (2023). Filters and highlighting in Power BI reports - Power BI. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/power-bi/create-reports/power-bi-reports-filters-and-highlighting.

Appendix A - Team Reflection

This reflection outlines our team's collaborative journey during these 3 weeks in building a mini Big Data platform. We successfully navigated the complexities of the Hadoop ecosystem, transitioned data into a serving layer, and visualized insights to answer critical stakeholder questions.

Team Roles & Individual Contributions

We divided responsibilities based on the project's core architectural layers to ensure specialized focus and efficiency:

- **Project Lead & Business Analyst (Ajmirah Eadally):** Defined the problem, identified stakeholders, drafted 3–4 primary questions, and established KPIs.
- **Data Engineer (Rukshar Baichoo):** Managed HDFS ingestion, folder structure, and authored Pig/MapReduce ETL scripts for the curated data zone.
- **Query Engineer (Christa Mungada):** Implemented MongoDB, designed schema and indexes, KPIs into aggregation pipelines, and exported results for BI analysis.
- **BI & Visualization Analyst (Kavina Narrainen):** Conducted deeper insight analysis, built the interactive dashboard (six visuals, two filters), and drafted recommendations.

Project Execution & Collaboration

Our workflow followed a structured approach, utilizing professional tools to manage our three-week timeline:

- **Agile Management with Jira:** We used a Jira Kanban board to track tasks from "To Do" to "Done". This allowed us to visualize progress and ensure no bottlenecks occurred
- **Version Control via GitHub:** All code was managed through a GitHub repository.
- **Communication (Teams):** Held regular meetings for brainstorming, requirement discussion, and handling raw vs. curated data zones.